

✓ Lab 03: Exploring Neural Information Processing

Course: ITAI-4374 — Neuroscience as a Model for AI Systems

Student: Duc Nguyen

Lab type: Individual

This notebook serves as both the **implementation + visualizations** and the **written report** for Lab 03.

✓ Setup

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
```

✓ Part 1: Biological Neuron

Resting potential -70 mV, threshold -55 mV. When input current drives the membrane above threshold, a spike ($+40$ mV) fires and the neuron enters a refractory period. This captures the **all-or-none** nature of real action potentials without full Hodgkin–Huxley machinery.

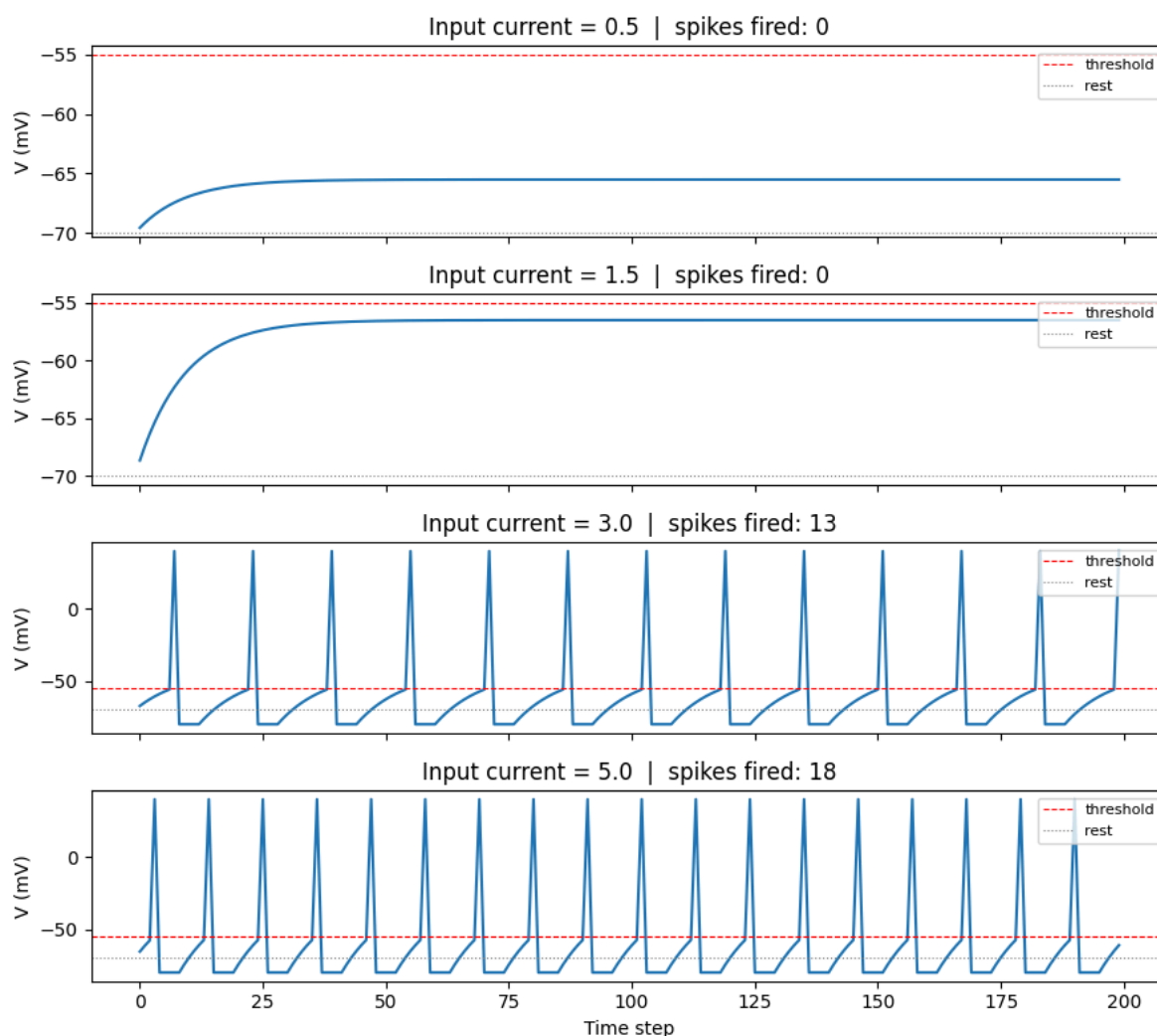
```
class BiologicalNeuron:
    """Simplified leaky integrate-and-fire neuron."""
    def __init__(self):
        self.resting_potential = -70.0
        self.threshold = -55.0
        self.peak = 40.0
        self.hyperpolarization = -80.0
        self.leak = 0.1
        self.refractory_steps = 5

    def simulate_response(self, input_current, time_steps=200):
        V = self.resting_potential
        trace, spikes = [], []
        refractory = 0
        for t in range(time_steps):
            if refractory > 0:
                V = self.hyperpolarization
                refractory -= 1
            else:
                V += input_current
                V += (self.resting_potential - V) * self.leak
                if V >= self.threshold:
                    trace.append(self.peak)
                    spikes.append(t)
                    refractory = self.refractory_steps
                    V = self.hyperpolarization
                    continue
                trace.append(V)
        return np.array(trace), spikes
```

✓ Response to varying input currents

```
neuron = BiologicalNeuron()
currents = [0.5, 1.5, 3.0, 5.0]
fig, axes = plt.subplots(len(currents), 1, figsize=(9, 8), sharex=True)
for ax, I in zip(axes, currents):
    trace, spikes = neuron.simulate_response(I)
    ax.plot(trace, color="#1f77b4")
    ax.axhline(-55, color="red", ls="--", lw=0.8, label="threshold")
    ax.axhline(-70, color="gray", ls=":", lw=0.8, label="rest")
    ax.set_ylabel("V (mV)")
    ax.set_title(f"Input current = {I} | spikes fired: {len(spikes)}")
    ax.legend(loc="upper right", fontsize=8)
axes[-1].set_xlabel("Time step")
```

```
plt.tight_layout(); plt.show()
```



Observations. Subthreshold current (0.5) leaks back to rest — no firing. As current increases, firing frequency rises. Each spike has the same amplitude (all-or-none).

✓ Part 2: Artificial Neuron

Weighted sum + bias, passed through a nonlinear activation. No time axis — a pure input→output function.

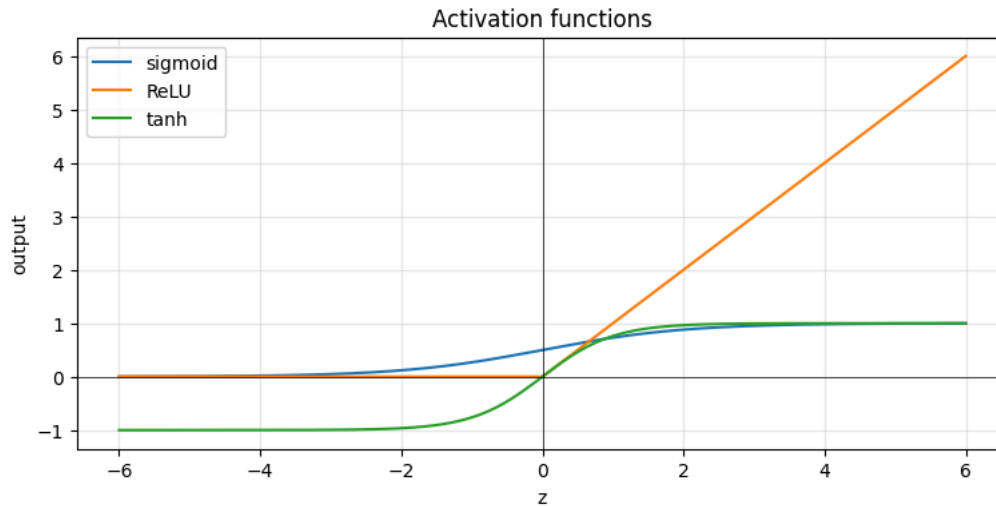
```
class ArtificialNeuron:
    def __init__(self, num_inputs, activation="sigmoid"):
        self.weights = np.random.randn(num_inputs)
        self.bias = 0.0
        self.activation = activation

    def activate(self, inputs):
        z = np.dot(self.weights, inputs) + self.bias
        if self.activation == "sigmoid":
            return 1 / (1 + np.exp(-z))
        if self.activation == "relu":
            return max(0.0, z)
        if self.activation == "tanh":
            return np.tanh(z)
        raise ValueError(self.activation)
```

✓ Comparing activation functions

```
x = np.linspace(-6, 6, 400)
plt.figure(figsize=(9, 4))
plt.plot(x, 1/(1+np.exp(-x)), label="sigmoid")
plt.plot(x, np.maximum(0, x), label="ReLU")
plt.plot(x, np.tanh(x), label="tanh")
plt.axhline(0, color="k", lw=0.5); plt.axvline(0, color="k", lw=0.5)
plt.title("Activation functions"); plt.xlabel("z"); plt.ylabel("output")
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

```
for act in ["sigmoid", "relu", "tanh"]:
    n = ArtificialNeuron(3, activation=act)
    print(f"{act:8s} -> {n.activate(np.array([0.5, 0.2, 0.8])):.4f}")
```

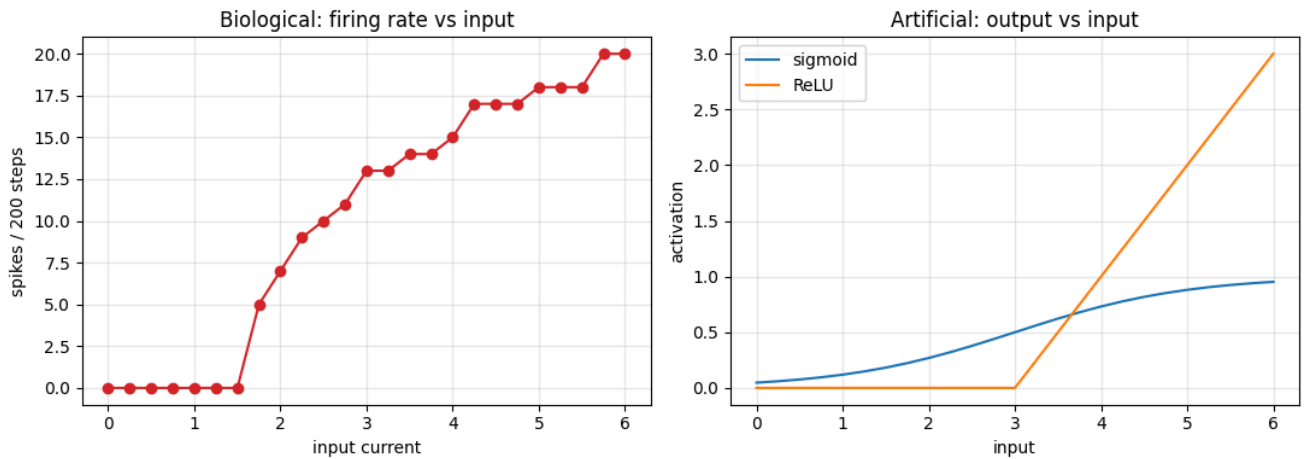


```
sigmoid -> 0.6767
relu    -> 0.5274
tanh    -> 0.5135
```

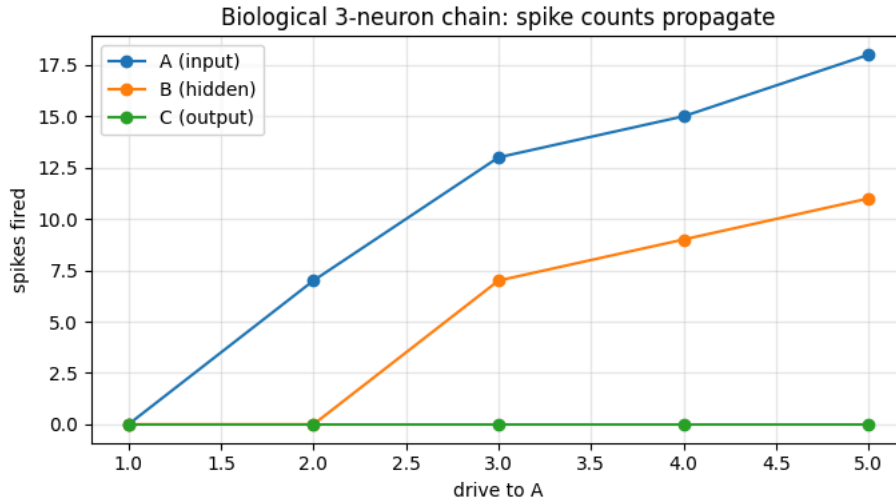
✓ Biological vs artificial: input-output curves

```
bio = BiologicalNeuron()
inputs = np.linspace(0, 6, 25)
rates = [len(bio.simulate_response(I)[1]) for I in inputs]
art_sig = [1/(1+np.exp(-(i-3))) for i in inputs]
art_relu = [max(0, i-3) for i in inputs]

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].plot(inputs, rates, "o-", color="#d62728")
ax[0].set_title("Biological: firing rate vs input")
ax[0].set_xlabel("input current"); ax[0].set_ylabel("spikes / 200 steps"); ax[0].grid(alpha=0.3)
ax[1].plot(inputs, art_sig, label="sigmoid")
ax[1].plot(inputs, art_relu, label="ReLU")
ax[1].set_title("Artificial: output vs input")
ax[1].set_xlabel("input"); ax[1].set_ylabel("activation"); ax[1].legend(); ax[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()
```



```
for i, lbl in enumerate(["A (input)", "B (hidden)", "C (output)"]):
    plt.plot(drives, rows[:,i], "o-", label=lbl)
plt.title("Biological 3-neuron chain: spike counts propagate")
plt.xlabel("drive to A"); plt.ylabel("spikes fired")
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()
```



Observations. The artificial network varies smoothly with inputs (differentiable units). The biological chain carries information as spike counts, with each stage compressing or amplifying depending on proximity to threshold. Differences: continuous vs discrete, instantaneous vs temporal, stateless vs refractory.

Written Analysis

1. Information processing: biological vs artificial neurons

Biological and artificial neurons share a common high-level template — integrate inputs, apply a nonlinearity, produce an output — but they differ sharply in the details. A biological neuron is a **dynamical system**: its membrane potential evolves continuously in time, integrating synaptic currents, leaking toward rest, and occasionally crossing a threshold that triggers a stereotyped action potential followed by a refractory period. Information is encoded not in the amplitude of any single spike (all spikes look alike — the all-or-none principle) but in **when and how often** spikes occur. Rate coding, temporal coding, and population coding all emerge from this substrate.

An artificial neuron, by contrast, is a **stateless function**. Given inputs, it returns a single real number — the weighted sum pushed through sigmoid, tanh, or ReLU. There is no time axis, no refractory period, no spontaneous activity, and no history. The gain is that artificial neurons are differentiable everywhere they matter, which makes gradient-based learning (backpropagation) tractable across networks with billions of parameters. The cost is that a single artificial unit is a far thinner caricature of its biological counterpart than the shared diagram suggests.

The lab simulations make this concrete. The biological model exhibits a hard threshold: below about 1.0 input current the neuron is silent; above it, firing rate climbs roughly linearly. The sigmoid artificial neuron smooths that transition into an S-curve, while ReLU preserves the threshold-linear character more faithfully — one reason ReLU has become the default in modern deep learning.

2. How biology can inspire better AI

Several properties of biological neurons are largely absent from mainstream deep learning and represent promising research directions:

- **Temporal coding and spiking computation.** Spiking neural networks encode information in spike timing, enabling event-driven computation that is dramatically more energy-efficient on neuromorphic hardware (Intel Loihi, IBM TrueNorth). The human brain runs on ~20 watts; a comparable artificial model burns kilowatts.
- **Local learning rules.** Biological synapses adapt using local signals (Hebbian learning, STDP), whereas backpropagation requires a global error signal routed backward through the network — a mechanism with no clear biological analogue. Local rules could enable online, lifelong learning.
- **Sparsity and recurrence.** Cortical activity is extremely sparse; only a small fraction of neurons fire at any moment. Artificial networks that enforce sparsity tend to be more robust and interpretable. Recurrence, ubiquitous in cortex, is still underused compared to feedforward stacks.

- **Neuromodulation and attention.** Biological brains dynamically reweight circuits via neuromodulators (dopamine, acetylcholine). Attention mechanisms in transformers are a crude functional analogue and have driven much of the last decade's progress.

3. Limitations of current artificial neuron models

The standard artificial neuron ignores almost everything interesting about real neurons: dendritic computation (individual dendrites perform nonlinear operations before the soma sums them), diverse cell types, stochastic synaptic transmission, plasticity on multiple timescales, and the metabolic constraints that shape biological coding choices. As a result, modern deep networks need orders of magnitude more parameters, data, and energy than brains to solve comparable tasks, and they generalize poorly out-of-distribution.

Promising directions include **dendritic networks** with internal nonlinear compartments; **spiking networks** trained with surrogate gradients; **predictive-coding architectures** that replace backprop with locally computable prediction errors; and **hybrid neuromorphic systems** pairing conventional deep learning with spiking front-ends. Together they point toward artificial neurons that are less of a cartoon and more of a computational primitive worthy of the name.

Conclusion

Simulating even a cartoon biological neuron alongside a standard artificial one is clarifying. The artificial neuron is an abstraction chosen for mathematical convenience, not biological fidelity, and the choice has paid off handsomely — but the gap between the two models is also a map of where AI might still borrow profitably from neuroscience.